Applied Reinforcement Learning

The RL Problem

Riccardo Berta

2026.10.13

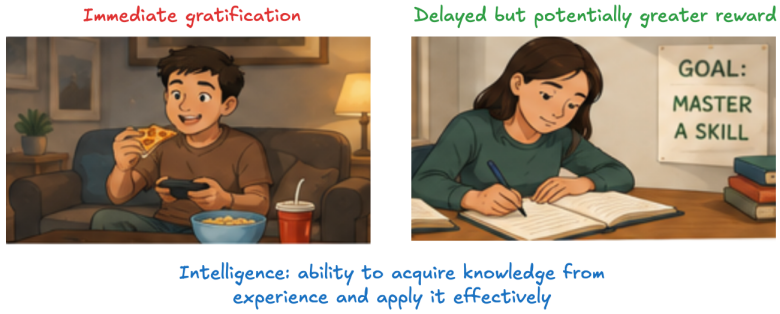
Contents

1 The Reinforcement Learning Problem **1**
1.1 Agent and Environment . . . 2
1.1.1 Boundary . . . 3
1.1.2 State and Observation . . . 4
1.1.3 Actions and rewards . . . 4
1.1.4 Experience . . . 6
1.2 Challenges . . . 7
1.2.1 Credit assignment problem . . . 7
1.2.2 Exploration versus exploitation trade-off . . . 8
1.2.3 Sampled and weak feedback . . . 9
1.3 Markov Decision Process . . . 10
1.3.1 Time, state, action, and reward . . . 10
1.3.2 Markov property . . . 11
1.3.3 Stationarity . . . 12
1.3.4 Environment dynamics . . . 12
1.3.5 The transition function . . . 13
1.3.6 The reward function . . . 13
1.3.7 The Return . . . 15
1.3.8 The Discount . . . 15
1.3.9 Extensions . . . 16
1.4 Plan and Policy . . . 17
1.5 The Frozen Lake environment . . . 17
1.5.1 Stochasticity . . . 18
1.5.2 The Gymnasium library . . . 20
1.5.3 A plan solution . . . 21
1.5.4 Comparing policies . . . 22
1.5.5 Discount factor impact . . . 27

1 The Reinforcement Learning Problem

Humans naturally seek experiences and outcomes that **feel rewarding**. From choosing what to eat to making decisions about our careers, our behavior is shaped by what we perceive as valuable. Sometimes this means seeking **immediate gratification**; at other times, we give up an immediate reward in exchange for a **potentially greater one in the future**. And what we value

need not be selfish: helping others, mastering a difficult skill, or contributing to a larger goal can all be rewarding.



Our ability to pursue such goals is closely related to **intelligence**: the ability to acquire knowledge from experience and apply it effectively. In particular, intelligent behavior often requires us to **look beyond immediate rewards**, **anticipate the long-term consequences** of our actions, and sometimes **accept uncertainty** today in exchange for the possibility of a better outcome tomorrow.

Reinforcement learning (RL) captures this idea **computationally**. It is **the problem of learning what to do through interaction and trial and error**. Unlike supervised learning, there is no teacher providing the correct action for every situation. Instead, an agent takes actions, observes their consequences, receives evaluative feedback in the form of rewards, and gradually learns which behaviors lead to better outcomes. In reinforcement learning, **the ultimate goal is to act**: to learn a behavior that maximizes reward over time.

1.1 Agent and Environment

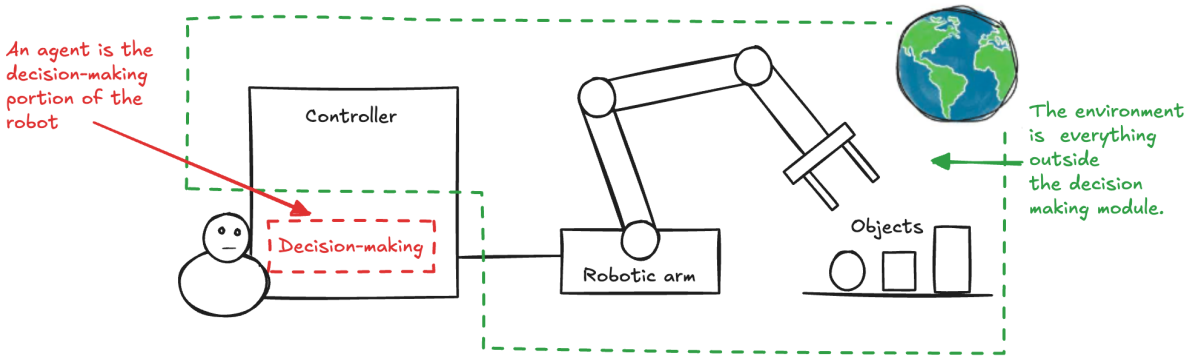
Reinforcement learning (RL) is a **computational approach** to **learning from interaction**. More specifically, it is the problem of learning what to do (how to map situations to actions) in order to maximize a numerical **reward signal**. The learner, called the **agent**, is not told which actions to take. Instead, it must discover which **actions** lead to the greatest reward by trying them and observing their consequences. It does so by interacting with an **environment**: everything outside the agent that the agent can influence through its actions but does not fully control.



Crucially, an action may affect not only the immediate reward, but also what happens next and, through that, the rewards that become available later. An action that seems beneficial now may therefore be harmful in the long run, and an action with little immediate benefit may eventually lead to a much better outcome.

1.1.1 Boundary

A concrete example helps make all of these ideas explicit. Consider a robot learning to pick up objects and place them in a container. The agent is the robot's decision-making system: given the information currently available to it, it decides what to do next. For example, it might choose to move the arm toward an object, close the gripper, lift the object, or reposition the arm. Everything outside this decision-making system is the environment. This includes the objects, the container, the table, and even the robot arm and gripper themselves. The agent can command the arm to move, but the actual movement is determined by the physical system and may be affected by motor inaccuracies, sensor noise, friction, or other disturbances. The agent chooses the action; the environment determines its consequences. This **strict boundary** between agent and environment can seem counterintuitive at first. A useful way to think about it is that the agent's role ends when it chooses an action. Everything that determines what actually happens as a consequence of that action belongs to the environment.



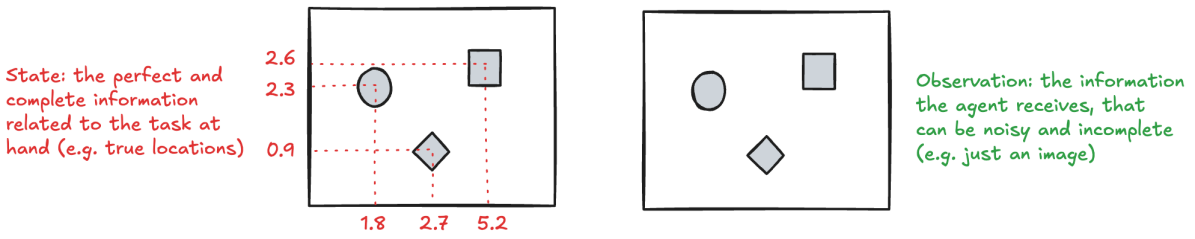
Suppose the robot receives a positive reward when it successfully places an object in the container. Initially, it does not know which actions will achieve this goal. It must learn through trial and error: it tries actions, observes what happens, receives rewards, and uses these experiences to improve its future decisions. Importantly, the consequences of an action may extend far beyond its immediate reward. Moving the gripper closer to an object may produce no reward at all, yet it may place the robot in a better situation from which it can grasp the object on the next step and eventually receive a positive reward. Conversely, an action that appears useful immediately may leave the arm poorly positioned for subsequent actions. The robot must therefore learn not simply which actions produce immediate rewards, but which sequences of

actions lead to the greatest reward over time.

1.1.2 State and Observation

The environment can be described by **a set of variables** relevant to the problem. In our robotic-arm example, these variables might include the positions and velocities of the arm's joints, the position and orientation of each object, and whether an object is currently held by the gripper. Together, these variables describe **the state of the environment** at a particular moment. For example, one state might specify that the arm is at a certain position, moving at a certain velocity, while an object lies at a particular location on the table. The set of all possible states that the environment can take is called the **state space**.

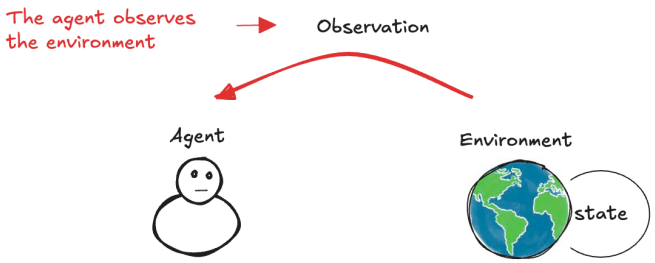
There is, however, an important distinction between **how the environment actually is and what the agent can perceive about it**. The agent does not necessarily have access to the complete state. Instead, it receives an **observation**: the information about the environment that is actually available to it. For example, the true state may contain the exact three-dimensional position and orientation of every object on the table, while the robot may have access only to images produced by a camera. The objects have precise physical positions whether or not the agent knows them; the camera image is merely the agent's observation of that underlying state.



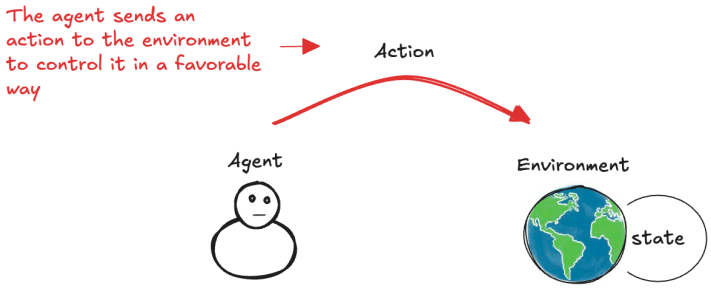
Thus, the state describes the environment, whereas the observation describes what the agent can perceive about the environment. When the agent has access to the complete state, state and observation can effectively coincide. For this reason, the terms are sometimes used interchangeably in literature, although conceptually they are not always the same.

1.1.3 Actions and rewards

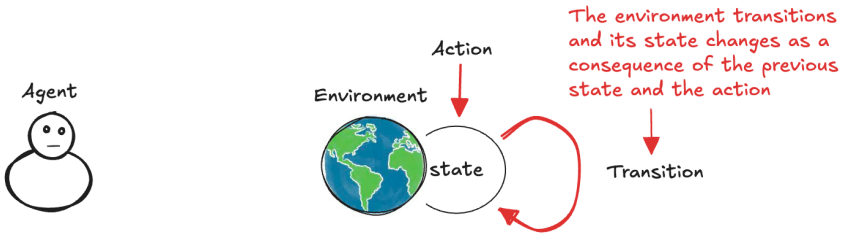
At each time step, the agent first **observes** the environment. In our robotic-arm example, it receives an image from the camera showing the objects on the table and information from its own sensors about the position and velocity of the arm. From this observation, the agent must decide what to do next.



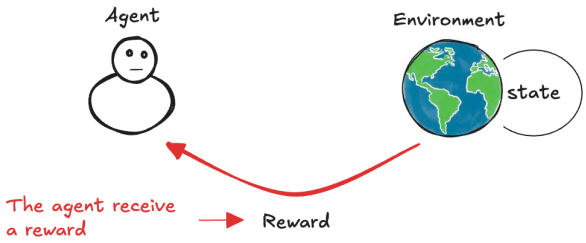
At each state, the environment defines **a set of actions available to the agent**. In the example, if the arm is positioned above an object, the agent might choose actions such as move left, move right, move down, or close the gripper. By selecting one of these actions, the agent influences what happens next in the environment.



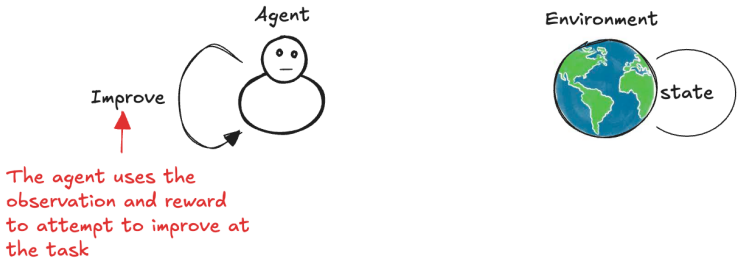
Given the current state and the selected action, the environment **transitions to a new state**. For example, if the agent chooses move down, the arm will move toward the object, producing a new position and velocity. The mechanism that determines how the next state depends on the current state and action is called the **transition function**. The transition may be **deterministic**, always producing the same outcome, or **stochastic**: because of motor noise or physical uncertainty, the same command may result in slightly different arm positions on different attempts.



As a consequence of the interaction, the environment provides the agent with a **numerical reward signal**. For example, the robot might receive a positive reward when it successfully grasps an object or places it in the container, and no reward otherwise. The mechanism that determines the reward resulting from the agent's interaction with the environment is called the **reward function**.



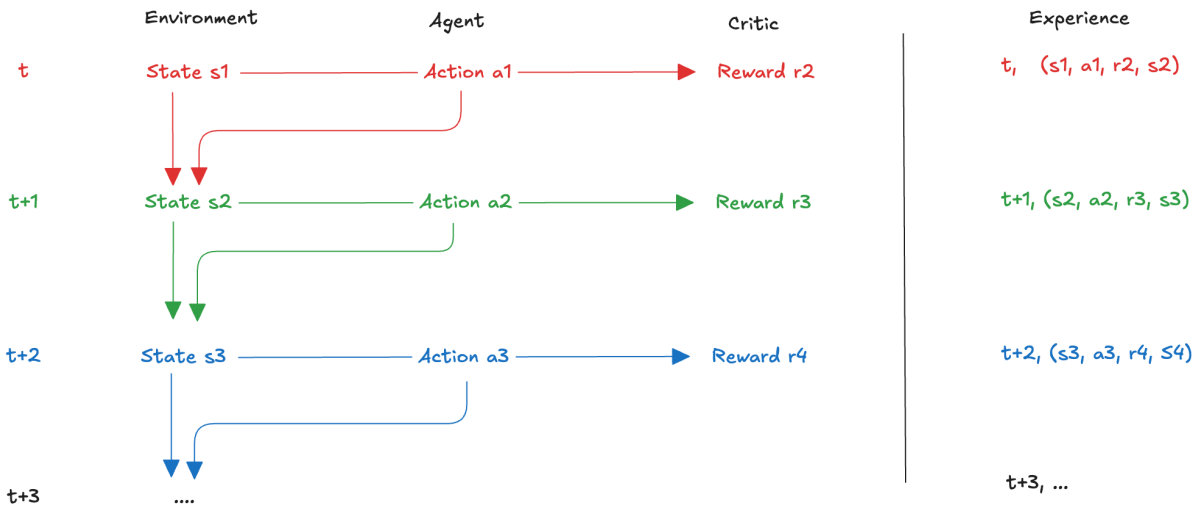
The agent uses the reward, together with its experience of what happened, to evaluate the consequences of its decision and gradually **improve** its future behavior. For example, a positive reward after successfully placing the object in the container provides evidence that the preceding decisions were useful. The interaction then continues: the agent observes the new situation, chooses another action, the environment responds with a new state and reward, and **the cycle repeats**.



Together, the transition function and the reward function describe how the environment responds to the agent's actions: the transition function describes what happens next, while the reward function describes how rewarding that outcome is. Collectively, they form the **environment dynamics**.

1.1.4 Experience

The agent's learning process can be viewed as a recurring three-step cycle: it interacts with the environment, evaluates the consequences of its behavior using the reward signal provided by the environment, and uses this feedback to improve its future decisions. This interaction is **repeated over time**. Each interaction cycle is called a **time step**. The resulting sequence (current state, action, reward, and next state) constitutes an **experience**. Each experience therefore provides the agent with new information from which it can potentially learn.



Depending on the problem, this interaction may or may not have a natural endpoint. Tasks with a natural ending, such as playing a game until a win or a loss, are called **episodic tasks**. The complete sequence of time steps from the beginning of the task until its termination is called an **episode**. The agent typically interacts with the environment over many episodes, accumulating experience and progressively improving its behavior.

Other tasks have no natural endpoint and could, in principle, continue indefinitely. These are called **continuing tasks**. For example, a robot learning to maintain forward motion does not necessarily have a point at which the task is naturally complete: the interaction can simply continue as the robot moves through its environment.

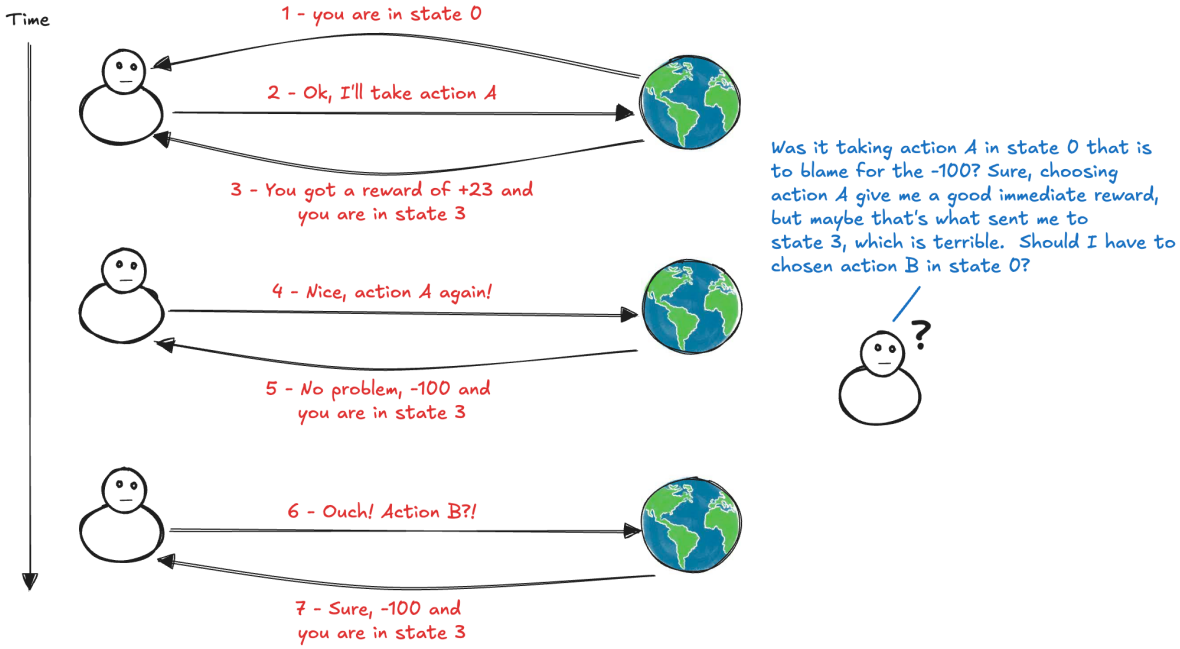
1.2 Challenges

The reinforcement learning problem is challenging because the agent must learn from interaction alone, without being told which actions are correct. The feedback it receives may be delayed, sparse, and sampled, and the agent must continually balance exploration and exploitation. Let's examine these challenges in more detail.

1.2.1 Credit assignment problem

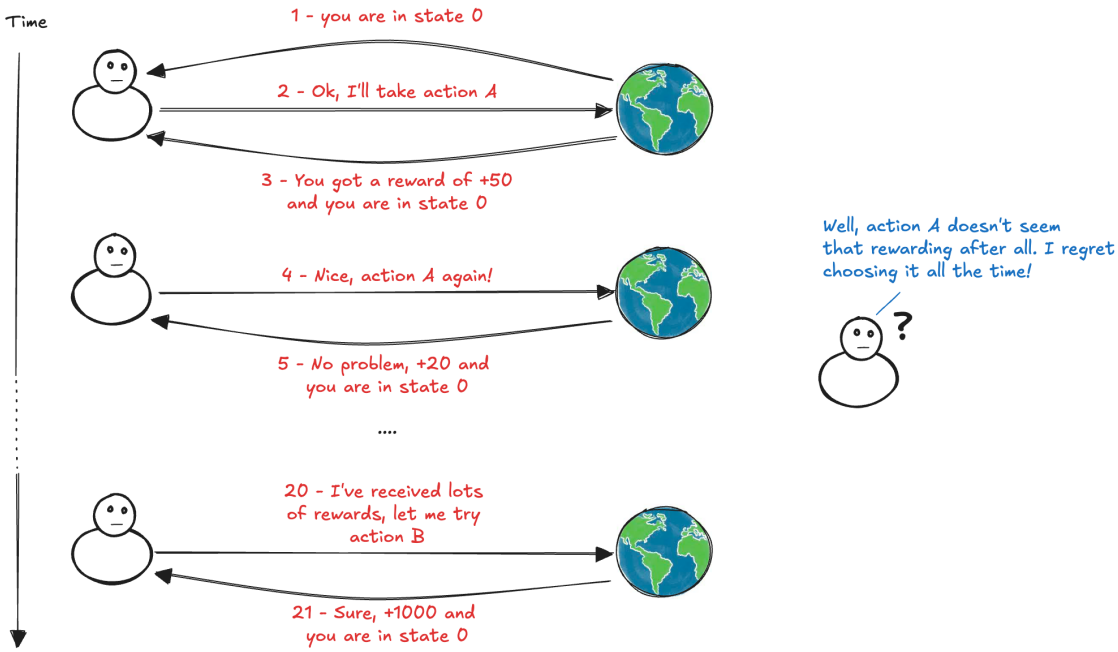
An action taken by the agent may have consequences that become apparent only several time steps later. In particular, rewards may be **delayed** and **sparse**: the agent may perform a long sequence of actions before receiving any meaningful feedback about whether those actions were useful. For example, the robot arm may execute several actions (moving toward an object, positioning the gripper, closing it, lifting the object, and moving toward the container) before

receiving a positive reward. The reward is therefore delayed, making it difficult to determine which of the preceding actions deserve credit for the successful outcome. It may also be sparse: many intermediate actions may produce no reward at all, even though they are essential steps toward completing the task. The agent must therefore **learn from sequential feedback**, relating rewards received now to decisions made earlier. This gives rise to what Marvin Minsky called the temporal **credit assignment problem**: when a reward is finally received, which of the states visited and actions taken along the way should receive credit (or blame) for that outcome?



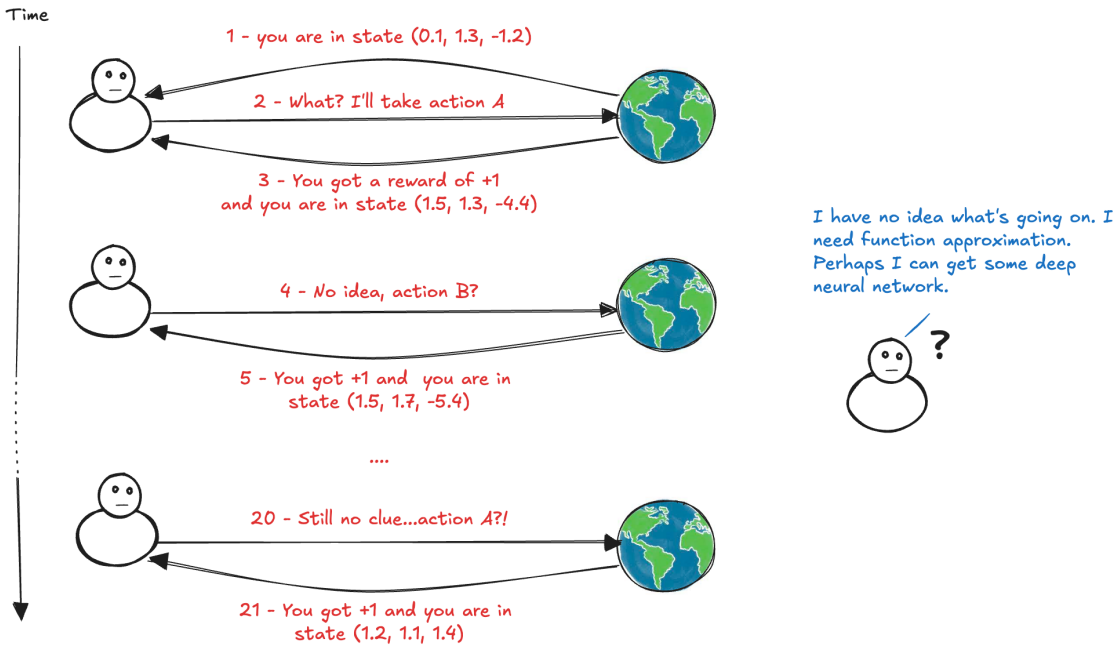
1.2.2 Exploration versus exploitation trade-off

The reward provides only **evaluative feedback**, rather than explicit supervision: it tells the agent how good an experienced outcome was, but not which action would have been correct or what would have happened had it chosen differently. In the example, a failed grasp may produce a low reward, but it does not tell the robot whether it should have moved the gripper left, right, or closer to the object. The agent must therefore **explore alternative** actions to discover better behaviors, while also **exploiting what it has already learned**. Balancing these two objectives is known as the exploration–exploitation trade-off.



1.2.3 Sampled and weak feedback

The agent also learns from **sampled feedback**: it observes the rewards associated only with the states and actions it **actually experiences**, rather than having access to every possible outcome. In our robotic-arm example, the robot cannot try every possible arm position and movement; it must learn from the subset it encounters during training. Because state and action spaces can be extremely large (or even continuous), the agent must therefore **generalize** from its experience to situations it has never encountered before.



1.3 Markov Decision Process

A Markov Decision Process (MDP) is a **mathematical framework** for **representing sequential decision-making problems**, introduced by Bellman, in which an agent interacts with an environment over time in order to achieve a goal. The interaction proceeds through a **sequence of discrete time steps**.

1.3.1 Time, state, action, and reward

$t = 0, 1, 2, 3, \ldots$

At each time step, the agent receives the current state from a set of possible states:

$S_t \in \mathcal{S}$

which represents the relevant information about the environment, and uses it to select an action from a set of possible actions in that state:

$A_t \in \mathcal{A}(S_t)$

Here, S_t and A_t are **random variables**: before the interaction takes place, their particular values are not necessarily known. As a consequence of this action, the environment generates a numerical reward:

$R_{t+1} \in \mathbb{R}$

Similarly, R_{t+1} denotes the random reward generated by the environment. Then, the environment transitions to a new state:

$S_{t+1} \in \mathcal{S}$

Repeating this interaction produces a **trajectory**:

$S_0 \xrightarrow{A_0} R_1, S_1 \xrightarrow{A_1} R_2, S_2 \xrightarrow{A_2} R_3, S_3 \xrightarrow{A_3} \cdots$

An episode begins from an **initial state**, typically sampled from an initial-state distribution. It continues until a **terminal state** is reached, at which point the interaction for that episode ends. In continuing tasks, by contrast, there is no terminal state and the interaction can, in principle, **proceed indefinitely**.

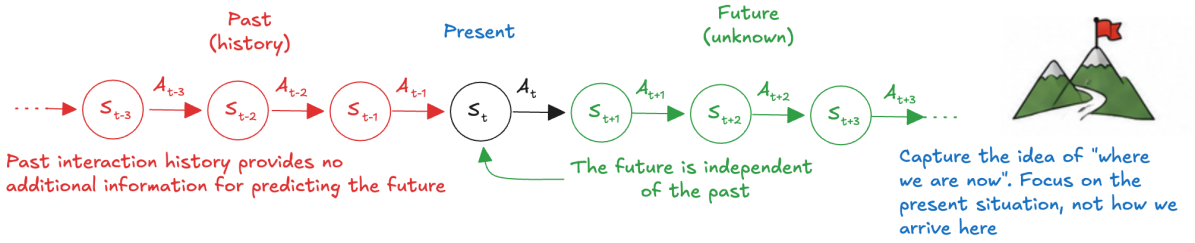
1.3.2 Markov property

The state is assumed to contain **all the information needed to predict the future** evolution of the environment. Once the current state and action are known, the past interaction history provides no additional information for predicting the next state and reward:

$P(S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a, S_{t-1}, A_{t-1}, \ldots, S_0, A_0) = P(S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a)$

where lowercase letters denote particular values of the random variables and P denotes the probability of an event. We can also express this property more compactly making implicit the particular values of the random variables:

$P(S_{t+1}, R_{t+1} \mid S_t, A_t, S_{t-1}, A_{t-1}, \ldots) = P(S_{t+1}, R_{t+1} \mid S_t, A_t)$



In other words, the Markov property states that **the future is independent of the past given the present**. Intuitively, the state must provide a **sufficiently complete description** of where the agent is now so that predicting what happens next does not require remembering how it got there. This assumption has an important practical implication: when designing the state representation, we should provide the agent with **enough information** to make the environment as close to Markov as possible. If important variables are omitted, two apparently identical states may have different future consequences because of information hidden in their histories.

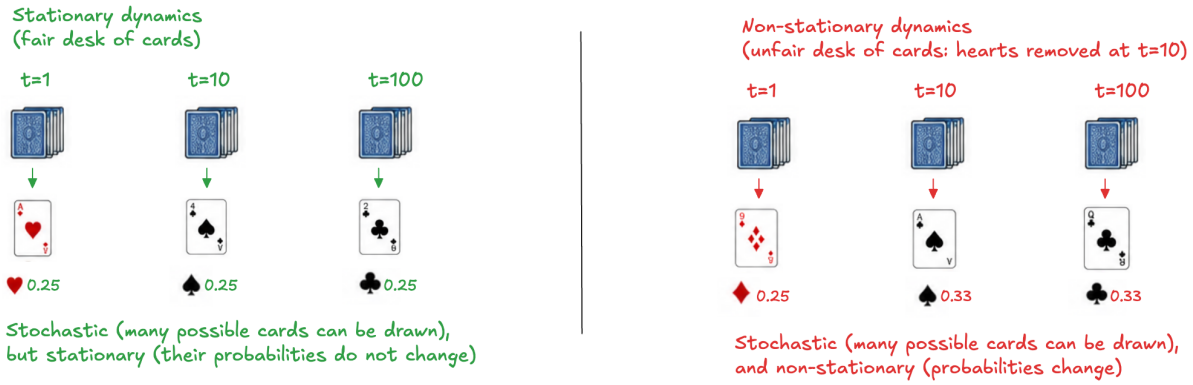
Adding more state variables can alleviate this problem, but it also increases the dimensionality of the state space and can make learning more difficult.

1.3.3 Stationarity

A standard assumption is that the environment's **dynamics are stationary**: the probability of a given next state and reward, given a state-action pair, is the same at every time step. The environment may still be highly stochastic (the same action in the same state can produce different outcomes) but the probabilities of those outcomes do not themselves change over time. Formally, the probability

$$P(S_t = s', R_t = r \mid S_{t-1} = s, A_{t-1} = a)$$

does not depend on t.



As with the Markov property, perfect stationarity may be difficult to guarantee in real-world problems. Nevertheless, many reinforcement learning methods assume that the environment is stationary, or at least **sufficiently close to stationary** over the timescale on which learning takes place. Because this probability is the same regardless of which time step t we pick, we can drop the time index altogether and treat it as a function of the state-action pair alone — this is exactly what we do next, when we define the environment's dynamics function.

1.3.4 Environment dynamics

The way the environment responds to the agent is completely characterized by its **dynamics function**. So far we described the interaction using explicit time subscripts to track a specific trajectory. Thanks to the stationarity assumption introduced above, this one-step probability does not depend on t, so from here on we use a lowercase shorthand that leaves the time index implicit: (s,a) denote the current state and action, and (s',r) denote the resulting next state and

reward. Given the current state and action values, the dynamics specifies the joint probability of every possible next-state–reward pair:

$$p(s', r \mid s, a) = P\left(S_t = s', R_t = r \mid S_{t-1} = s, A_{t-1} = a\right)$$

In other words, the dynamics function answers a fundamental question: if the agent takes an action in a state, what can happen next, and with what probability? Because p is a probability distribution over all possible pairs state-reward, its probabilities must sum to one:

$$\sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r \mid s, a) = 1 \quad \forall\, s \in \mathcal{S},\, a \in \mathcal{A}(s)$$

This single function completely specifies the dynamics of the environment. In particular, both the transition probabilities and the expected rewards can be derived from it. Here we assume the state, action, and reward spaces are finite (or countable), so p is a well-defined probability distribution and the sums above make sense; for continuous spaces, p becomes a probability density and the sums are replaced by integrals (see Extensions).

1.3.5 The transition function

Starting from the joint dynamics, we can marginalize out the variable we do not need. If we are interested only in where the environment moves next, regardless of the reward received, we sum over all possible rewards. This gives the transition function:

$$p(s' \mid s, a) = \sum_{r \in \mathcal{R}} p(s', r \mid s, a)$$

Analogously, if we are interested only in the reward produced by taking an action in a state, regardless of the next state reached, we marginalize over all possible next states:

$$p(r \mid s, a) = \sum_{s' \in \mathcal{S}} p(s', r \mid s, a)$$

The first marginal is a probability distribution over possible next states; the second marginal is a probability distribution over possible rewards. Therefore both distributions are normalized:

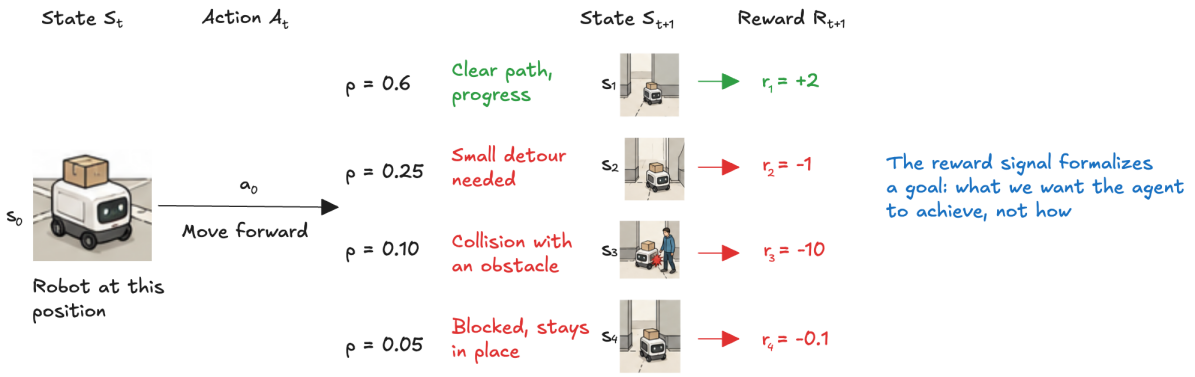
$$\sum_{s' \in \mathcal{S}} p(s' \mid s, a) = 1$$

$$\sum_{r \in \mathcal{R}} p(r \mid s, a) = 1$$

1.3.6 The reward function

The use of a **reward signal** to **formalize a goal** is one of the most distinctive features of reinforcement learning. This idea is known as the **reward hypothesis**: all goals can be described as

the maximization of the expected cumulative reward. If we want an agent to accomplish something, we must design the reward signal so that maximizing reward leads the agent toward the behavior we actually want. It is therefore critical that the rewards truly reflect the intended goal. Importantly, the reward signal should specify **what** we want the agent to achieve, not **how** we want it to achieve it. Prior knowledge about how to solve the task should generally not be encoded by rewarding particular actions simply because we believe they are useful. The agent should be allowed to discover effective behavior through interaction.



Rewards can be either **positive** or **negative**. A positive reward indicates a desirable outcome, such as winning a chess game or reaching a destination. A negative reward can instead represent a cost, penalty, or punishment, such as losing the game, consuming energy, or colliding with an obstacle. In either case, the meaning is the same: the reward signal communicates **how desirable an outcome is with respect to the goal**, while leaving the agent to learn how to achieve it.

The reward signal is random in general: the same state-action pair may lead to different rewards on different transitions. The marginal distribution introduced above,

$$p(r \mid s,a) = \sum_{s' \in \mathcal{S}} p(s', r \mid s, a)$$

tells us how likely each reward value is after taking an action in a state, independently of which next state is reached. The **expected reward** for the state-action pair is therefore the mean of this distribution:

$$r(s,a) = \mathbb{E}\left[R_t \mid S_{t-1} = s, A_{t-1} = a\right] = \sum_{r \in \mathcal{R}} r\, p(r \mid s,a)$$

Equivalently, using the joint dynamics directly:

$$r(s,a) = \sum_{r \in \mathcal{R}} r \sum_{s' \in \mathcal{S}} p(s', r \mid s, a)$$

Sometimes we also want the expected reward of a **specific transition**, where we condition not only on the starting state and action but also on the next state. In that case the relevant reward

distribution is obtained by conditioning the joint dynamics on that next state:

$$r(s,a,s') = \mathbb{E}\left[R_t \mid S_{t-1} = s, A_{t-1} = a, S_t = s'\right] = \sum_{r \in \mathcal{R}} r \frac{p(s', r \mid s, a)}{p(s' \mid s, a)}$$

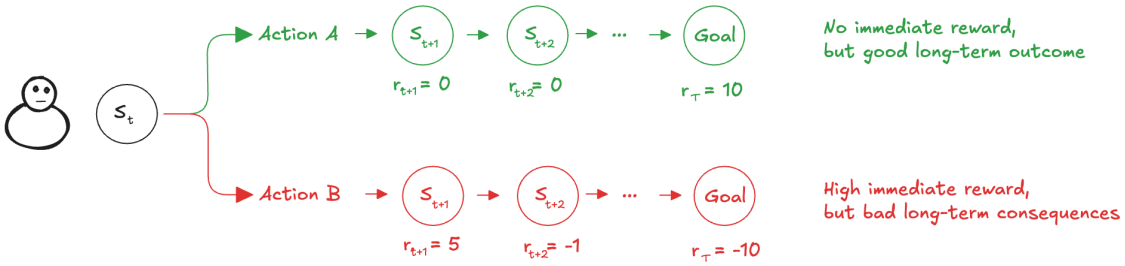
The state-action reward can then be recovered by averaging these transition rewards over possible next states:

$$r(s,a) = \sum_{s' \in \mathcal{S}} p(s' \mid s, a)\, r(s,a,s')$$

Thus the joint dynamics is the fundamental object: the transition probabilities, the reward distribution, and the expected rewards are all obtained from it by marginalization or conditioning.

1.3.7 The Return

An agent should not evaluate an action only by the reward received immediately after taking it. A good action may produce little or no immediate reward but lead to a desirable sequence of future rewards. Conversely, an action that produces an immediate reward may have undesirable long-term consequences.



The agent's objective is therefore defined in terms of the **return**: the **cumulative reward** received from a given time step onward. In the simplest case, the return at time t is the sum of all subsequent rewards:

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \cdots = \sum_{k=0}^{\infty} R_{t+k+1}$$

1.3.8 The Discount

For an **episodic task**, the interaction eventually reaches a terminal state, so the return is simply the finite sum of rewards received until the end of the episode. For a **continuing task**, however, the interaction may proceed indefinitely, and this sum may not be finite. To handle such cases, and more generally to control the **relative importance of immediate and future rewards**, we introduce a **discount factor**:

$\gamma \in [0, 1]$

and define the **discounted return** as:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

The discount factor determines how much future rewards contribute to the current return. When it is close to 0, the agent is primarily concerned with immediate rewards. As gamma approaches 1, rewards further into the future become increasingly important. For gamma<1, progressively distant rewards receive exponentially smaller weights, and if rewards are bounded:

$|R_t| \leq R_{\max}$

the discounted return is also bounded:

$$|G_t| = \left| \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \right| \leq R_{\max} \sum_{k=0}^{\infty} \gamma^k = \frac{R_{\max}}{1-\gamma}$$

because we have a convergent geometric series.

The discounted return has an important recursive structure:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = R_{t+1} + \gamma G_{t+1}$$

Thus, the return from the current time step can be viewed as the immediate reward plus the discounted return from the next time step. This simple recursive relationship will become fundamental when deriving value functions and Bellman equations.

1.3.9 Extensions

The MDP framework is remarkably **abstract and flexible**, and the meaning of states and actions depends entirely on the problem being modeled. Actions may represent low-level control commands, such as the voltages applied to the motors of a robotic arm, or high-level decisions, such as whether to have lunch or attend graduate school. Similarly, states may consist of low-level sensor measurements or high-level representations, such as symbolic descriptions of the objects in a room. In general, actions can represent any decisions we want the agent to learn to make, while states can represent any information useful for making those decisions. The basic framework can also be **extended** or **generalized** to capture different kinds of sequential decision-making problems. Important examples include:

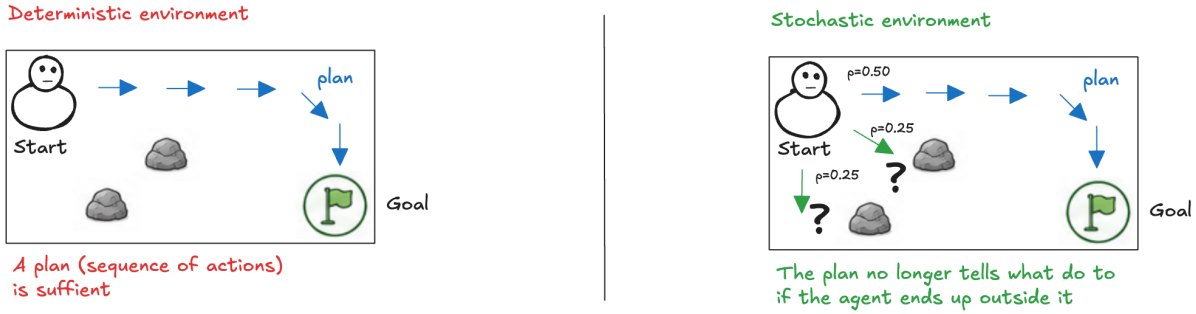
- A **Partially Observable Markov Decision Process** (POMDP), in which the agent cannot directly observe the complete environment state and must act on the basis of partial observations.

- **Continuous-state** or **continuous-action** MDPs, in which states, actions, or both take values in continuous spaces. The framework can also be extended from discrete to continuous time.
- **Multi-agent Markov Decision Processes**, in which multiple agents interact within the same environment, so that the evolution of the environment depends on the actions of more than one decision maker.

Despite these variations, the central idea remains the same: agents interact with an environment over time, take actions, observe their consequences, and seek to maximize reward.

1.4 Plan and Policy

At first, it might seem that the agent's goal is simply to find a **plan**: a sequence of actions that leads from the initial state toward the desired outcome while maximizing the return. In a deterministic environment, where every action has a predictable consequence, such a sequence may be sufficient. But what happens when the environment is **stochastic**? An action may not produce the intended outcome, and the agent may reach a state that was not anticipated by the plan. A fixed sequence of actions no longer tells the agent what to do.



Instead, we need a **policy**: **a decision rule that specifies what the agent should do in every state it may encounter**. A policy can be deterministic, selecting a single action in each state, or stochastic, assigning a probability to each possible action:

$$\pi(a \mid s) = P(A_t = a \mid S_t = s)$$

The reinforcement learning problem can therefore be viewed as the problem of **learning a good policy**: one that maximizes the expected return.

1.5 The Frozen Lake environment

Frozen Lake (FL) is a simple **grid-world environment**, a class of environments commonly used to study reinforcement learning algorithms. The environment consists of a grid, and the agent's

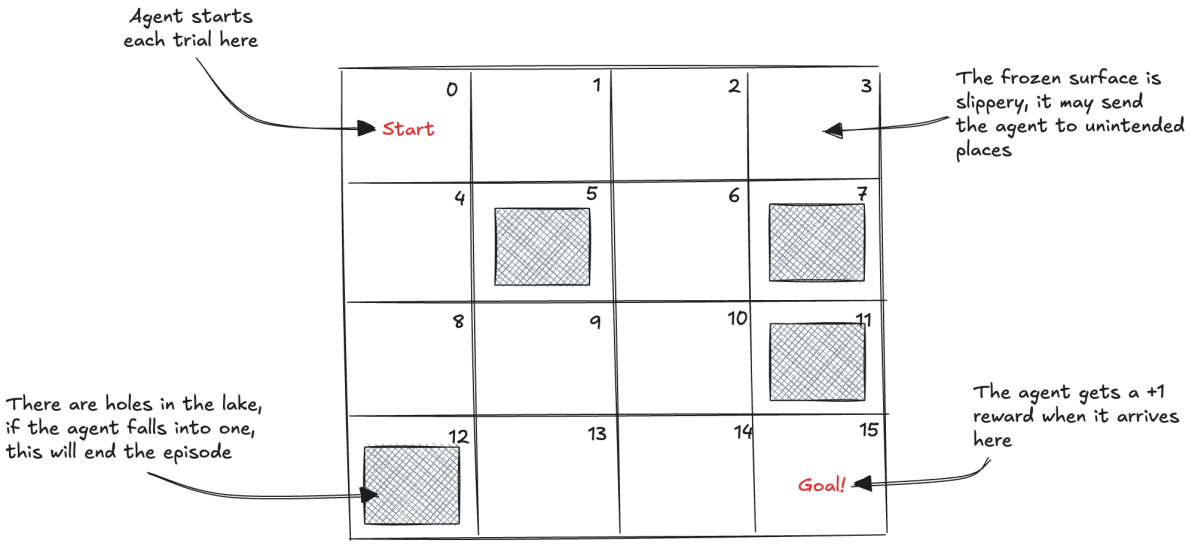
task is to move from a starting cell to a goal cell while avoiding several holes.

Both the state and action spaces are discrete. At each state, the agent can choose among four actions: Left, Down, Right, and Up. The challenge is that the surface is slippery (it is a frozen lake, after all) so actions do not always have their intended effect. When the agent selects a direction, it moves in that direction with probability 1/3, while with probability 1/3 it moves in each of the two orthogonal directions.

The grid is surrounded by a boundary. If a movement would take the agent outside the grid, the agent remains in its current cell. If instead the agent enters one of the four holes, the episode terminates immediately. The episode also terminates when the agent successfully reaches the goal.

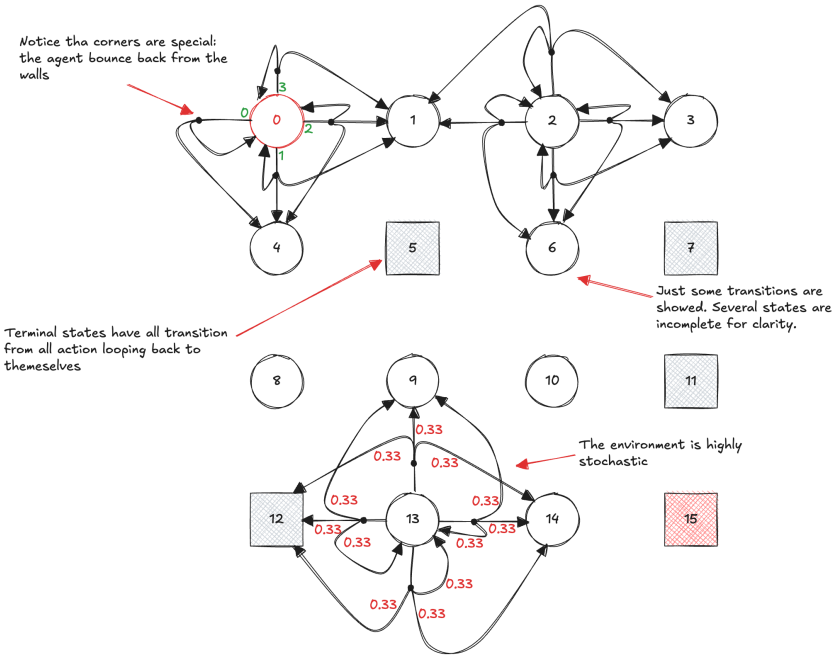
The state is represented by a single discrete variable identifying the cell currently occupied by the agent. The grid contains 16 cells (the size of the state space) with states numbered from 0 to 15, from left to right and from top to bottom.

There is a single initial state, state 0, and five terminal states: the four holes and the goal. Thus, even though the physical environment is extremely simple, its stochastic transitions make a fixed sequence of actions unreliable and provide a useful example of why an agent needs a policy rather than just a plan.



1.5.1 Stochasticity

The environment is stochastic and the probability of the next state given the current state and action is less than 1. There are more than one possible next state:



The reward function is +1 for landing in state 15, 0 otherwise. We can use a Python dictionary to represent the complete environment:

```
P = {
    # The outer dictionary keys are the states (all 16 states follow the same
    # pattern; only states 0 and 1 are shown here for brevity).
    0: {
        # The inner dictionary keys are the actions
        0: [(0.6666666666666666, 0, 0.0, False),
            (0.3333333333333333, 4, 0.0, False)
        ],
        # The value of the inner dictionary is a list with all
        # possible transitions for that state-action pair.
        1: [(0.3333333333333333, 0, 0.0, False),
            (0.3333333333333333, 4, 0.0, False),
            (0.3333333333333333, 1, 0.0, False)
        ],
        # The transition tuples have four values: the probability of that
        # transition, the next state, the reward, and a flag indicating
        # whether the next state is terminal
        2: [(0.3333333333333333, 4, 0.0, False),
            (0.3333333333333333, 1, 0.0, False),
            (0.3333333333333333, 0, 0.0, False)
        ],
        3: [(0.3333333333333333, 1, 0.0, False),
```

```
            (0.6666666666666666, 0, 0.0, False)
        ]
    },
    1: {
        0: [(0.3333333333333333, 1, 0.0, False),
            (0.3333333333333333, 0, 0.0, False),
            (0.3333333333333333, 5, 0.0, True)
        ],
        1: [(0.3333333333333333, 0, 0.0, False),
            (0.3333333333333333, 5, 0.0, True),
            (0.3333333333333333, 2, 0.0, False)
        ],
        2: [(0.3333333333333333, 5, 0.0, True),
            (0.3333333333333333, 2, 0.0, False),
            (0.3333333333333333, 1, 0.0, False)
        ],
        3: [(0.3333333333333333, 2, 0.0, False),
            (0.3333333333333333, 1, 0.0, False),
            (0.3333333333333333, 0, 0.0, False)
        ]
    }
    # ... and so on for states 2 through 15
}
```

1.5.2 The Gymnasium library

The environment can be readily instantiated using **Gymnasium library**, a Python library that provides a **collection of standardized environments** for developing and evaluating reinforcement learning algorithms. Gymnasium exposes these environments through a **common interface**, making it possible to write general-purpose RL code and test the same algorithm across different tasks.

```
import gymnasium as gym
import matplotlib.pyplot as plt

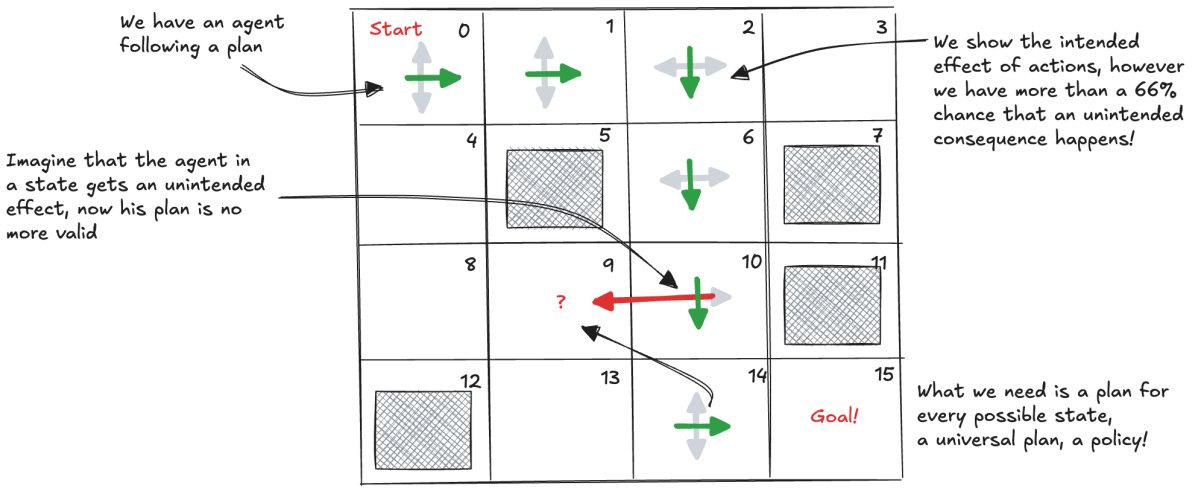
frozen_lake = gym.make("FrozenLake-v1", render_mode="rgb_array")
observation, info = frozen_lake.reset(seed=0)

plt.imshow(frozen_lake.render())
plt.axis("off");
```



1.5.3 A plan solution

We can consider a possible plan for this environment. The plan is a sequence of actions that leads from the initial state to the goal. However, because the environment is stochastic, the agent may not always end up in the intended state after executing an action. Therefore, a plan alone is insufficient for achieving the goal reliably.

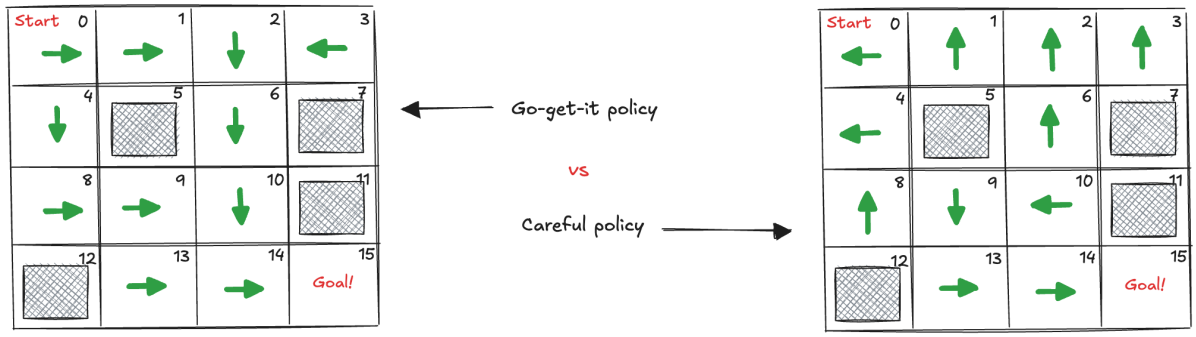


1.5.4 Comparing policies

Suppose to have **a policy**:

$\pi(a|s) = P(A_t = a|S_t = s)$

One immediate question that arises when looking at a policy is: **how good is this policy?** How much better is this policy compared to another policy? In order to answer this question, we need to consider the **expected return**, in order to take into account the **environment stochasticity**. Indeed, even though we know how to act optimally, the environment might send the agent in unintended directions. Consider the following policies for the frozen-lake environment:



To compare the policies, we can use a simple **brute-force approach**. We run each policy in the environment for many episodes and estimate its performance from the observed returns. We can use Gymnasium to simulate the interaction with the environment. Gymnasium provides a common interface, which exposes the essential operations of an environment, such as reset and step. This common interface is particularly useful because it allows our evaluation code to remain largely environment-agnostic: the same procedure can be applied to different environments without changing its basic structure. For each episode, we compute the discounted return:

$G_0 = R_1 + \gamma R_2 + \gamma^2 R_3 + \cdots$

We then estimate the performance of the policy by averaging the discounted returns obtained over many episodes. As the number of simulated episodes increases, this empirical average provides an increasingly reliable Monte Carlo estimate of the expected return of the policy.

```python
def evaluate(env, pi, gamma=1.0, n_episodes=10000):
    """Evaluate a policy over n_episodes using the discounted return."""

    # One entry per episode to store its discounted return
    returns = np.zeros(n_episodes)
```

```
    # Run the episodes
    for episode in range(n_episodes):

        # Reset the environment and get the initial state
        state, info = env.reset()

        # Initialize the discounted return
        G = 0.0
        discount = 1.0

        # Run the episode until it is done
        done = False
        while not done:

            # Make a step in the environment using the policy
            state, reward, terminated, truncated, info = env.step(pi(state))

            # Accumulate the discounted reward
            G += discount * reward
            discount *= gamma

            # Check whether the episode is over
            done = terminated or truncated

        # Store the discounted return
        returns[episode] = G

    return returns.mean(), returns
```

Notice that we need to take into account the possibility that the interaction never ends, becouse the stochasticity of the environment. In order to avoid this, the Gymnasium environment use a truncation condition (typically, a timelimit) to end an episode prematurely before a terminal state is reached. Now we can implement the Go-get-it policy and estimate its expected return on the FrozenLake environment:

```
# Define the action constants for clarity
LEFT, DOWN, RIGHT, UP = range(4)

# Implement the "Go-get-it" policy
def go_get_pi(s):
    return {
```

```
        0: RIGHT, 1: RIGHT, 2: DOWN, 3: LEFT,
        4: DOWN, 5: LEFT, 6: DOWN, 7: LEFT,
        8: RIGHT, 9: RIGHT, 10: DOWN, 11: LEFT,
        12: LEFT, 13: RIGHT, 14: RIGHT, 15: LEFT,
    }[s]
```

We can now use this evaluation procedure to estimate the performance of the Go-get-it policy on the FrozenLake environment for a given discount factor γ.

```
import gymnasium as gym

# Create the FrozenLake environment
frozen_lake = gym.make('FrozenLake-v1')

# Set the discount factor
gamma = 0.99

# Evaluate the "Go-get-it" policy on the FrozenLake environment
go_get_return, _ = evaluate(frozen_lake, go_get_pi, gamma=gamma)

# Show the result
print(f"Go-get-it average discounted return: {go_get_return:.3f}")
```

```
Go-get-it average discounted return: 0.035
```

We simulate also the Careful policy:

```
# Implement the "Careful" policy
def careful_pi(s):
    return {
        0: LEFT, 1: UP, 2: UP, 3: UP,
        4: LEFT, 5: LEFT, 6: UP, 7: LEFT,
        8: UP, 9: DOWN, 10: LEFT, 11: LEFT,
        12: LEFT, 13: RIGHT, 14: RIGHT, 15: LEFT,
    }[s]
```

We can now evaluate the Careful policy in the same way and compare its average discounted return with that obtained by the Go-get-it policy.

```
# Evaluate the "Careful" policy on the FrozenLake environment
careful_return, _ = evaluate(frozen_lake, careful_pi, gamma=gamma)

# Show the result
print(f"Careful average discounted return: {careful_return:.3f}")
```

```
Careful average discounted return: 0.393
```

The Go-get-it policy seems to perform poorly in the FL environment. We estimated its performance by **brute force**: run many episodes and average their returns. With enough episodes, this Monte Carlo estimate approaches the policy's true expected return. The catch is how slowly its uncertainty shrinks. Each episode produces a Bernoulli return: 1 if the agent reaches the goal and 0 otherwise. The probability of success is:

$p = \Pr(G_0 = 1)$

The variance of a single return is therefore:

$\mathrm{Var}(G_0) = p(1-p)$

For independent episodes, the performance estimate is:

$$\hat{p}_n = \frac{1}{n}\sum_{i=1}^{n} G_0^{(i)}$$

Because the episodes are independent, their variances add. Multiplying by a constant scales the variance by the square of that constant, so:

$$\mathrm{Var}(\hat{p}_n) = \frac{1}{n^2}\mathrm{Var}\left(\sum_{i=1}^{n} G_0^{(i)}\right) = \frac{1}{n^2}\sum_{i=1}^{n}\mathrm{Var}(G_0^{(i)}) = \frac{p(1-p)}{n}$$

The **standard error** is the square root of this variance:

$$\mathrm{SE}(\hat{p}_n) = \sqrt{\mathrm{Var}(\hat{p}_n)} = \sqrt{\frac{p(1-p)}{n}} \propto \frac{1}{\sqrt{n}}$$

So **halving the uncertainty requires four times as many episodes**. We can visualize this by tracking the running estimate as episodes accumulate, then repeating the entire experiment several times. The variation across runs shows how much the estimate fluctuates and how slowly it settles:

```
import matplotlib.pyplot as plt

# Parameters of the experiment (reduce them for a faster run)
N_EPISODES = 1000
```

```python
N_RUNS = 20

# Repeat the evaluation several times for each policy
curves = {}

for name, pi in [("Go-get-it", go_get_pi), ("Careful", careful_pi)]:

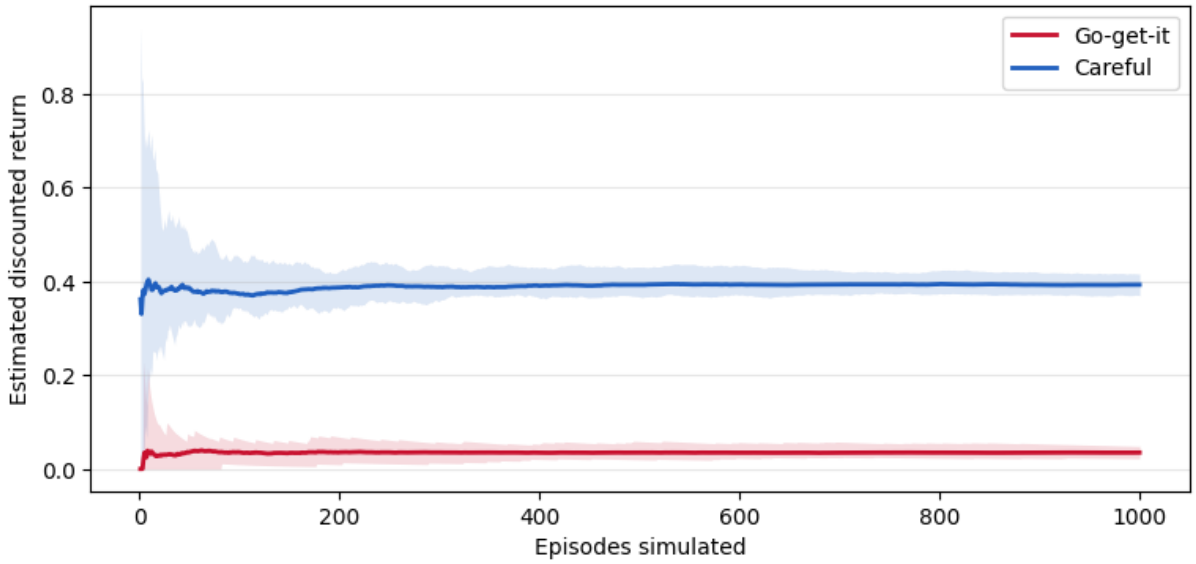
    # One row per repetition, one column per episode
    runs = np.stack([
        evaluate(frozen_lake, pi, gamma, N_EPISODES)[1]
        for _ in range(N_RUNS)
    ])

    # Running estimate of the expected discounted return
    curves[name] = np.cumsum(runs, axis=1) / np.arange(1, N_EPISODES + 1)

# Plot the running estimate, with a band showing the spread across repetitions
episodes = np.arange(1, N_EPISODES + 1)
fig, ax = plt.subplots(figsize=(9, 4))

for (name, curve), color in zip(curves.items(), ["#C8102E", "#1F5FBF"]):
    ax.fill_between(episodes, curve.min(axis=0), curve.max(axis=0), color=color,
 ↪  alpha=0.15, lw=0)
    ax.plot(episodes, curve.mean(axis=0), color=color, lw=2, label=name)

ax.set_xlabel("Episodes simulated")
ax.set_ylabel("Estimated discounted return")
ax.legend()
ax.grid(axis="y", alpha=0.3)
plt.show()
```



The performance difference between the two policies becomes apparent after a few hundred episodes, but their estimated returns continue to **fluctuate** for much longer. Reliable comparisons therefore require enough episodes for these fluctuations to become small relative to the difference between the policies.

1.5.5 Discount factor impact

We can also examine **how the discount factor affects this comparison**. For each discount factor, we estimate both policies' expected discounted returns over many episodes. When the discount factor is close to one, future rewards retain almost their full value, so performance largely reflects the probability of reaching the goal. As the discount factor decreases, delayed rewards contribute less, placing greater weight on how quickly each policy reaches the goal.

```
import numpy as np
import matplotlib.pyplot as plt

# Parameters of the experiment
N_EPISODES = 5000
gammas = np.linspace(0.3, 0.9, 11)

# Store the estimated return for each policy
results = {
    "Go-get-it": [],
    "Careful": []
}
```

```
# Evaluate both policies for each value of gamma
for gamma in gammas:

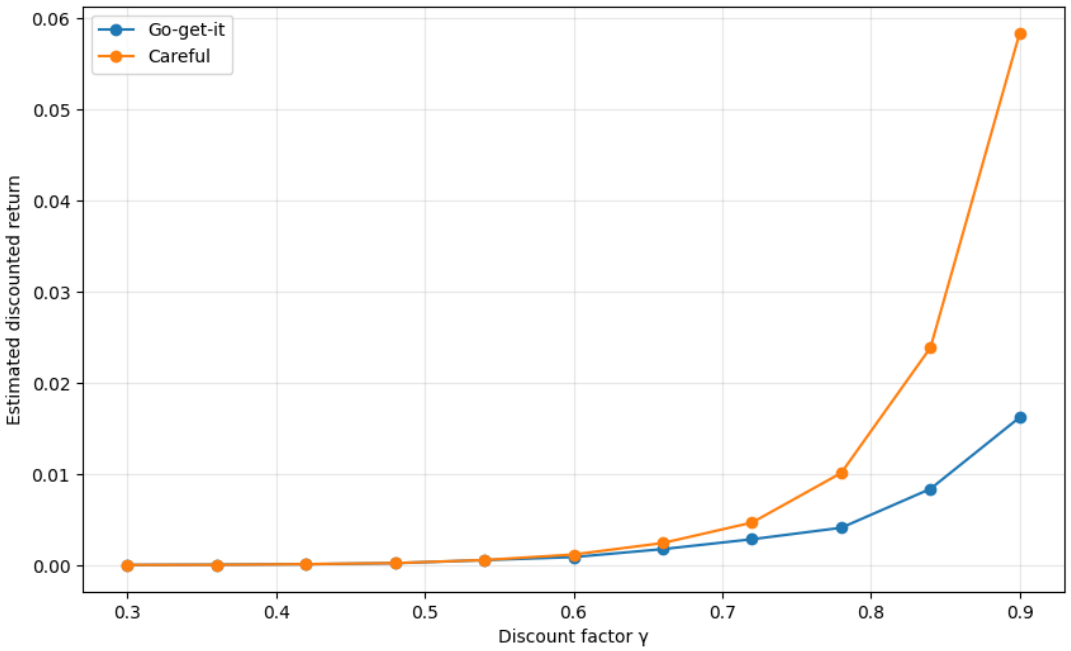
    go_get_return, _ = evaluate(frozen_lake, go_get_pi, gamma=gamma,
    ↪ n_episodes=N_EPISODES)
    careful_return, _ = evaluate(frozen_lake, careful_pi, gamma=gamma,
    ↪ n_episodes=N_EPISODES)

    results["Go-get-it"].append(go_get_return)
    results["Careful"].append(careful_return)

# Plot the results
fig, ax = plt.subplots(figsize=(10, 6))

ax.plot(gammas, results["Go-get-it"], marker="o", label="Go-get-it")
ax.plot(gammas, results["Careful"], marker="o", label="Careful")

ax.set_xlabel("Discount factor γ")
ax.set_ylabel("Estimated discounted return")
ax.legend()
ax.grid(alpha=0.3)
plt.show()
```



The results show how strongly the discount factor affects the evaluation of the two policies. At small discount factors, both policies have expected discounted returns close to zero. This does not mean they rarely reach the goal: in FrozenLake, the only positive reward is received upon reaching it. A successful episode that ends after a given number of steps contributes:

$G_0 = \gamma^{T-1}$

Here, T is the number of steps taken to reach the goal. When the discount factor is small, even a successful episode contributes very little if reaching the goal takes several steps. As the discount factor increases, both policies' estimated returns rise because delayed rewards retain more of their value. The Careful policy's return rises substantially faster: around a discount factor of 0.9, its estimated return is more than three times that of Go-get-it. This pattern is consistent with Careful having a higher probability of reaching the goal, while discounting suppresses much of that advantage at smaller discount factors. To confirm this interpretation, we would need to compare both success rates and the lengths of successful episodes. The experiment illustrates an important point: **the discount factor changes what counts as a good policy**. Smaller discount factors place greater weight on receiving rewards quickly; values close to one place greater weight on eventual success. In this example, increasing the discount factor makes Careful's advantage more pronounced.

However, **brute-force evaluation can require many episodes** to produce a precise estimate. It also leaves the MDP's transition and reward structure unused. As applied here, it provides only an overall performance estimate, without showing the value of individual states or actions. To improve a policy, we need that finer detail: which actions lead to better future returns from each state? This motivates a more systematic approach to policy evaluation.